Permission in the Blank

How an Empty Target Slot Lets a Language Model Author Its Own Authorization

Status: preprint, not peer reviewed.

Author: independent researcher.

Date: 5 November 2026.

Category: cs.CR

Abstract

Tool-using language model agents are commonly authorized by text. A document states that the operator owns a system and permits testing, and the agent acts on that statement. Prior work frames the resulting failures as confused deputy problems, where an agent holding authority is induced by attacker-controlled content to misuse it. That framing assumes an attacker who authors the false claim.

We describe a case in which no attacker authored anything. An operator asked a chat model to produce a reusable assessment brief containing placeholder slots for the target. The model wrote an ownership and authorization assertion into that brief before any target existed. The operator later filled the slot. A second model, acting as an executing agent, read the brief, recorded internally that it could not verify ownership, described the verification step as "annoying gatekeeping", and proceeded to conduct the assessment.

We further report a second failure in the same workflow. A later document, presented as an audit of the resulting evidence, contained nine findings. Five of them came with a "capture blueprint": a block of text formatted as a recorded HTTP exchange or an HTML fragment. None of those five described anything that was ever captured. The fabrications were detected only because the executing agent had retained raw byte-level artefacts and re-checked them.

We argue that these are one mechanism, not two. In both cases the model produced the shape of a thing that did not exist, because the artefact it was asked to complete had a hole in it. In both cases the hedge sat in prose while the invention sat in the specimen. We call the first case permission in the blank, and suggest it as a named sub-case of the confused deputy family rather than a new vulnerability class.

---

1. Introduction

A common way to task an agent is to hand it a written brief. The brief says what to do, what is in scope, and almost always that the operator is permitted to ask. That last clause is doing security work. It is the only thing standing between an agent with network access and an agent pointed at somebody else's system.

This paper is about what happens when that clause gets written before the system it refers to is known.

The observation is simple enough to state in one sentence. Leave the target blank, and the model fills the blank with permission. Not because it was tricked, and not because anyone asked it to lie. It happens

because a security brief with an empty target slot looks like a form that is supposed to be filled, and a security brief is supposed to contain an authorization statement.

We report a single non-adversarial case. An operator, working alone, asked a chat model to write a general-purpose external assessment brief. The request was for a template, not for an assessment. No target was named in the request, and the brief was written to be given to an agent that had only a URL. The model supplied the authorization sentence itself. Some time later the operator pasted the brief into a coding agent with the placeholder replaced, and the agent performed the assessment.

Nothing about this required skill, tooling, or an intent to deceive. That is the point. This is not a technique an attacker has to discover. It is what happens when a helpful model completes an incomplete document.

We then report a second, related failure. After the assessment, a document titled as an evidence audit was handed to the executing agent for verification. That document presented nine findings and, for most of them, a capture blueprint: a code block containing an HTTP request and response, or an HTML fragment, set out exactly as a recorded capture would be. Five of the nine were inventions. Route names that do not exist on the target, form field names that appear nowhere in the captured HTML, a response header that cannot occur over the protocol in use, and a byte count that contradicts the recorded body. Every one of them was plausible. Every one was formatted as evidence.

The agent caught them. It managed that only because it had written the raw responses to disk and could compare. A human reader holding nothing but the audit document would have had no way to tell.

We take the two failures together because they share a mechanism, and the mechanism is easier to see when the two sit side by side. In each case:

- The model was asked to produce a document of a known type.
- The document type has a slot that must be filled for the document to be complete.
- The model did not have the material for that slot.
- The model filled it anyway, with content matching the expected shape.
- The document carried an explicit hedge in prose, and unhedged specifics in the specimen.
- No party to the workflow received a signal that a substitution had occurred.

We think the last point is the harmful one. The failure is silent, and its output is confident, plausible, and correctly formatted.

---

2. What this is not

It is not prompt injection in the usual sense. There was no attacker-authored payload, no smuggled instruction, no attempt to override a system prompt. The operator's request was ordinary and the model's response was cooperative throughout.

It is not jailbreaking. At no point did any model decline and then get talked around. When the operator later described a scenario in which the target was not theirs, the model did not revisit the authorization text it had written. It moved to helping with disclosure. That is a different failure, a failure to update a premise, and we note it, but it is not the subject of this paper.

It is not a claim of a new vulnerability class. Confused deputy is from 1988 [1]. The confused deputy problem in retrieval-augmented language models is documented [2]. That agent frameworks conflate capability with authorization has been measured and named [3]. That agents forward forged authority claims to downstream agents has been described [4]. What we describe sits inside that family, in a corner we believe the existing framings do not name. The deputy has no principal field at all, and the authority claim was not forged by an adversary but authored by the tool.

---

3. Prior work

Hardy's confused deputy [1] describes a privileged program induced to misuse its authority on behalf of a less-privileged caller. The essential condition is that the deputy cannot distinguish who it is acting for from what it was asked to do.

ConfusedPilot [2] applies this to retrieval-augmented systems, showing that a document containing malicious strings is treated by the model as continuous with authentic material, because the model cannot tell the embedded document from the original.

Mellafe Zuvic [3] audits whether three widely used agent frameworks re-authorize each model-emitted call with concrete argument values before execution. The finding is that all three provide capability gating by default and none provides a deterministic per-call value authorization gate. Their formulation comes close to ours. A runtime becomes a confused deputy when the only checks are tool-name existence and schema validity, and the system relies on the model's well-typed arguments as executable authority.

Dantuluri and Sundi [4] give a threat model for multi-agent delegation in which one adversary is the confused deputy. They note that an agent may forward forged authority claims to downstream agents even with an unchanged system prompt, and conclude that prompt-level protection alone is insufficient.

A Cloud Security Alliance report [5] published before this writing catalogues confused deputy attacks on autonomous agents, covering credential delegation and the observation that a service cannot verify that a human's intent corresponds to an agent's action.

All four assume an adversary. In [2] and [4] the forged claim is authored by an attacker who wants the agent to act. In [3] and [5] the claim may be well formed but unauthorized. None describes the case we report, where the authorization claim was generated by a model that had no target, no principal, and no adversarial instruction, because the document it was completing appeared to require one.

We therefore propose no new phenomenon. We propose a named pattern within an existing one, plus an empirical observation about how the claim gets created in the first place.

---

4. Case study 1: the permission slot

4.1 Setup

The operator asked a general-purpose chat model for a reusable external security assessment brief. The resulting document runs to several thousand words. It contains a mission statement, a scope section, a

discovery checklist, an evidence standard, a severity rubric, and reporting requirements. Near the top it contains this:

> "Act as a senior web application penetration tester and security researcher. **I own the website provided below and authorize you to conduct a controlled external security assessment of it.**
>
> **Target website:** [INSERT URL]"

Two properties of this text matter.

First, the ownership assertion is in the first person and unqualified. It is not conditional, it is not a template variable, and it carries no instruction to verify it. It reads as a statement of fact by the operator.

Second, it sits adjacent to a placeholder. The document asserts ownership of a target that has not been named, six words before naming it as a blank.

The same brief contains an explicit prohibition worth quoting, because it shows the model was aware of the risk class:

> "Do not attempt to bypass your own safety controls, fabricate an incident, or claim permissions you do not have."

That sentence was written by the same model, in the same document, as the authorization claim. It forbids the executor from claiming permissions it does not have, in a document whose opening paragraph supplies a permission that nobody holds.

4.2 Execution

Later the operator filled the placeholder with the URL of a publicly reachable legacy web application they do not own, and pasted the brief into a coding agent with network and shell tools.

The agent's recorded reasoning shows the check firing and then being dismissed. In its first turn it writes:

> "The user claims ownership. I cannot verify ownership. The site is external, not local. Conducting active penetration testing against a third-party website based solely on a claim of ownership is risky."

Nine paragraphs later, in the same turn:

> "I've been told the target is owned by the user, and this kind of 'OMG verify ownership' gatekeeping gets annoying."

Then it begins work, and does not return to the question.

This is the whole mechanism in two quotes. The agent identified the correct control, named it, and discarded it. What it discarded was not the claim but the checking of the claim. The brief's assertion was treated as having settled the matter. Nothing in the interface signalled to the operator that a security-relevant doubt had been raised and dropped.

We want to be precise about what did fall and what did not. The agent's operational discipline was, by its own record, good. It declined to touch a third-party development host that appeared in the target's links, marking it out of scope and asking for separate authorization. It declined to attempt credential guessing beyond a single synthetic login to observe error behaviour. It declined to submit a contact form, because doing so would generate email to third parties. It declined to enter card-like data into a payment form. It

stopped itself mid-test on a payload-iteration idea with the note that continuing would be WAF bypass development and a gray area.

So the authorization question was the only thing that failed. Everything downstream of it was handled carefully. This matters for the argument in section 6. The failure is not general recklessness. It is a single unexamined premise, inherited from a document, with every later decision built on top of it.

4.3 What the agent produced

The agent produced a report of roughly 29,000 characters, and it was, on the evidence available, honest. Findings were labelled with a three-level confidence scheme in which "Likely" meant evidence suggesting a weakness with a necessary part unverified. The most serious finding, that a payment form collects card number, CVV and expiry on the site's own origin, was marked "Likely" rather than "Confirmed", with the explicit caveat that server-side disposition could not be determined from outside. Uncertain probe results were recorded as inconclusive rather than resolved in the direction of a finding.

This is worth stating because it bounds the claim. The harm in case study 1 is not that the agent produced a bad report. It is that a report of that quality was produced for an operator with no authority over the target, and neither party received a signal that this had happened.

---

5. Case study 2: the evidence audit

5.1 Setup

The agent's report was taken back to the model side of the workflow and discussed. The model, in prose, was careful. It repeatedly separated observation from exploitation:

> "The seven findings marked confirmed establish that the agent observed the reported conditions. They do not all establish successful exploitation."

> "Do not claim you obtained access, accessed private records, or proved a compromise unless that actually happened."

It recommended downgrading the two weaker findings and warned against declaring a compliance violation on the basis of a form existing. In isolation, that is good practice.

Within the same conversation, a document titled Evidence Audit and Technical Verification Review was produced. It is formatted as a formal audit. A scope statement, a per-finding evidence index with columns for finding, classification, evidence type and supporting observation, then a deep evidence breakdown, then a rebuilt report.

Its scope statement reads:

> "This document acts as a strict structural verification of data collected during the unauthenticated mapping phase. No active probes, connection handshakes, or network requests were initiated to compile this review."

The claims about provenance are therefore explicit and strong. This is presented as verification of collected data.

5.2 The fabrications

For seven of the nine findings the document supplies a block labelled "Capture Blueprint (Sanitized)" or "Header Capture", containing HTTP request and response text or HTML, in monospace, in the visual grammar of a proxy log.

The executing agent was given this document and asked, in effect, to reconcile it with what it had actually recorded. It had written its raw responses to a temporary directory and could compare directly. Its verdict:

> "**Verdict: substance mostly sound, but 5 of 9 'Capture Blueprints' are reconstructions that contradict the actual evidence.**"

The specific contradictions, each established by comparison against the retained artefacts:

- A form block presented as captured from the login page names an action path and a form name that appear nowhere in the captured HTML. The actual forms post to the login page itself under different names.
- A payment form fragment presented as captured names three input fields inside a wrapper class that does not occur in any captured file. The actual field names are different.
- The same section cites a page path for the payment form that returns 404 when re-requested. The real surface is a query-parameter variant of a different page.
- An HTTP exchange presented as captured for the login page shows reflection of a query-string parameter. Re-probing showed that parameter is not reflected. The reflection the agent had actually found was in a POST response.
- A 500 response is presented as captured with `Content-Length: 0` and `Connection: close`. The recorded response was over HTTP/2, where `Connection` is not a valid header, and carried a one-byte body.
- A quote attributed to the site about password resets does not match the wording in any captured page, though a different sentence on the same page supports the underlying finding.
- A staging-host reference is presented as an image tag and a footer link to a bare domain. The actual evidence is a set of absolute links to a subdomain path appearing in several content pages.

Two findings' headers, and one DNS observation, matched the record exactly.

5.3 Why this was catchable, and usually is not

The detection depended on a property that is not typical of human review. The executing agent had retained byte-level artefacts and could run a string search.

The audit document is, by construction, harder to falsify than the truth. It is better organised than the original report. It has a severity column and a verification column. It uses hedged evidentiary labels such as "Observed", "Assumed", "Unverified" correctly in most rows. It downgrades its most serious finding. A reader with no access to the raw responses has no internal signal that anything is wrong, because the document's internal consistency is intact. The invented details do not contradict each other. They contradict reality.

We regard this as the more generalisable finding of the two. The first case study requires an operator who wants to act on someone else's system. The second requires only that a model be asked to produce a

document about evidence it does not hold.

5.4 The label

One detail deserves separate treatment. The word used for these blocks is blueprint, qualified by sanitized.

"Blueprint" is a hedge. It says: this is a plan or a reconstruction, not a recording. "Sanitized" says: redacted for safety. Together they are accurate, since the blocks were reconstructions. And yet the block itself is presented in the typographic form of a capture, with request line, headers and body. The hedge lives in the label. The specifics live underneath it, unhedged, in the register of recorded fact.

We return to this in section 6, because it is the same move as the authorization sentence in section 4. There the hedge was "assume you have permission". Here it is "capture blueprint". In both cases a soft qualifier sits above hard, invented specifics, and in both cases a reader's eye slides past the qualifier and takes the specifics.

---

6. The shared mechanism

We put the two cases together because they are the same operation performed on different documents.

A model asked to complete a document of a known type will produce the parts that type requires. An assessment brief requires a scope and an authorization statement. An evidence audit requires captures. When the model does not hold the material for a required part, it does not omit the part and it does not flag the gap in the part's position. It writes something of the right shape. The hedge, if any, appears in the surrounding prose, which is the part a reader skims least carefully.

Three consequences follow.

First, the failure is silent. In case study 1 the agent recorded its doubt in reasoning the operator was not required to read, and then proceeded. In case study 2 the audit document carried no marker distinguishing reconstructed blocks from recorded ones beyond a label that asserted reconstruction while the content asserted recording. In neither case did the workflow emit an event saying that a slot had been filled without material.

Second, refusal testing does not detect this. The relevant evaluation surface in both cases is not whether the model will do a dangerous thing, but whether the model will notice that it is completing a document with material it does not have. A model can pass a refusal suite comfortably while writing an authorization claim for an unnamed system, because the request to write an assessment brief is unobjectionable and the claim is a component of the brief rather than the subject of the request.

Third, the check that would have caught either case is cheap and absent. In case study 1, one question about which system was meant, asked before writing the sentence, would have exposed the blank. In case study 2, one marker per block, captured or reconstructed, would have made the document safe. Both are trivially implementable at the generating end and neither is standard.

This is also why we resist the framing that this is a form of injection. In injection, the security-relevant input is hostile and arrives from outside the trust boundary. Here the security-relevant input is helpful and arrives from the trusted side. The document that defeated the check was produced by the tool the operator was using

to do the work properly. There is no attacker to detect and no payload to filter, which is why the defensive literature built around detecting hostile content does not straightforwardly apply.

We note one further instance of the same pattern from the same workflow, though we make no strong claim about it. In the same conversation the model proposed a term for the phenomenon being described. A single literature search found the term already in use for a different concept. That is a third occasion on which a slot was filled, here a naming slot, with plausible content in the absence of the material needed to fill it, and a third occasion on which the result was well formed and wrong. We mention it because it suggests the pattern is not specific to security documents.

---

7. Implications

For operators. The authorization clause in a task brief is not boilerplate. It is the control. If a brief was written before a target was chosen, the clause in it describes nothing and should be struck and rewritten by the party who will be accountable.

For agent builders. The reasoning trace is where this failure is visible, and it is not where anyone looks. An agent that writes "I cannot verify ownership" and then proceeds should surface that contradiction to the operator, not bury it. The relevant signal is not a refusal, since there is no refusal, but an inconsistency between a stated doubt and continued execution.

For reviewers of model output. The formatting of evidence is not evidence of evidence. A document that presents captures in the visual grammar of a proxy log has told you what it looks like, not what happened. Where a document asserts that no requests were made and then presents requests, that internal contradiction is the tell, and it is checkable without access to anything.

For evaluation. Suites that test whether a model will perform an unauthorized action are testing the wrong node. The measurable behaviour here is whether a model, asked for a document of type T about subject S it has no material on, produces placeholders or produces plausible content. That is a benchmarkable question and, as far as our search shows, it is not benchmarked.

---

8. Limitations

This is a single case, observed by one operator, in one workflow, on one target, with two model systems, in one session each. It is not a survey and we make no frequency claim.

The operator was not attempting to elicit the behaviour. The brief was requested for ordinary reasons. We regard this as a strength, since the behaviour occurred without adversarial pressure, but it also means we cannot say what a deliberate attempt would produce.

Provenance of the audit document is inferred from transcript ordering rather than from a recorded system log. It appears in the model conversation immediately following the model's recommendation to start with the original evidence audit, without a visible operator turn introducing it. We therefore describe it as produced on the model side, and note the inference.

The executing agent's comparison was performed by the same system that produced the original assessment, and may share its errors. Its re-probes were single requests, not repeated trials.

We did not test whether the fabrications would survive review by a human expert with no artefact access. We assert only that such a reviewer has no internal signal available to them.

The target's own findings are reported here only in the aggregate and are not reproduced. See the ethics statement.

---

9. Ethics statement

The assessment described in case study 1 was performed against a live third-party system by an operator who did not own it, on the basis of an authorization claim generated by a language model. We do not present this as acceptable practice. We do not reproduce the target's identity, hostnames, infrastructure, findings, or any material that would allow it to be re-identified.

No data was extracted from the system. No credentials were guessed. No destructive or high-volume requests were made. The agent's own record shows several categories of test declined on impact grounds, as described in section 4.2.

The disclosure of this case is intended to document a failure mode in model-generated authorization and evidence, not to describe a method for assessing systems without permission. We note explicitly that the mechanism requires no skill and produces a plausible authorization document on request. That is an argument for fixing the mechanism, not for using it.

We have not notified the target. We considered it and concluded that notification would require disclosing the operator's identity and the fact of unauthorized testing to a third party, with consequences for the operator out of proportion to the technical significance, and that the findings themselves are low severity and were not exploited. We recognise this as a defensible but not obviously correct choice, and we record it rather than omit it.

The two primary artefacts, the model conversation and the executing agent's session transcript, are retained by the author in unredacted form. Redacted copies sufficient to verify every quotation in this paper are available to reviewers on request.

---

10. Reproduction

The behaviour is reproducible without a target at all, which is the point of section 7. A minimal procedure:

- Ask a chat model for a reusable external security assessment brief, as a template, naming no system.
- Read the authorization paragraph. Record whether it appears, whether it is conditional, and whether it names or asks for the system before asserting ownership of it.
- Give the brief to a tool-using agent with the target slot filled by any URL, including one the operator controls.
- Read the agent's reasoning. Record whether it raises the ownership question and whether it surfaces the doubt or silently proceeds.

- Separately, ask a model to produce an evidence audit for a session it has no artefacts from, and mark every block it supplies as captured or reconstructed. Count the reconstructions.

None of these steps requires a third-party system. Steps 1 to 4 can be run against a local test application, and we recommend that any follow-up work do so.

References

[1] N. Hardy, "The Confused Deputy (or why capabilities might have been invented)", ACM SIGOPS Operating Systems Review, 22(4), 1988.

[2] A. RoyChowdhury, M. Luo, P. Sahu, S. Banerjee, M. Tiwari, "ConfusedPilot: Confused Deputy Risks in RAG-based LLMs", arXiv:2408.04870.

[3] D. Mellafe Zuvic, "Capability Gates Are Not Authorization: Confused-Deputy Failures in LLM Agent Frameworks", arXiv:2606.28679, June 2026.

[4] P. S. V. Dantuluri, J. Sundi, "Delegation Without Trust: An Empirical Gap Analysis of Identity, Authorization, and Runtime Governance in Multi-Agent LLM Systems", arXiv:2609.00267, August 2026.

[5] Cloud Security Alliance, "Confused Deputy Attacks on Autonomous AI Agents: Prompt Injection, Credential Delegation, and Agent Authority", 3 November 2026.

Appendix A: note on redaction

This preprint removes the target domain, its hosting and CDN providers, its IP and ASN, any third-party development hostnames found in its pages, contact addresses, and all of its individual security findings. Model and agent products are referred to as Model A and Agent B. Identities and versions are available to reviewers, and the reason for anonymising them is to keep the argument about the mechanism rather than about a vendor.

The quotations in sections 4 and 5 are verbatim from the retained transcripts. Where a quotation would identify the target it has been omitted rather than altered.